

Recurrent Neural Networks and Language Models

Text and Web Mining (IS6751)

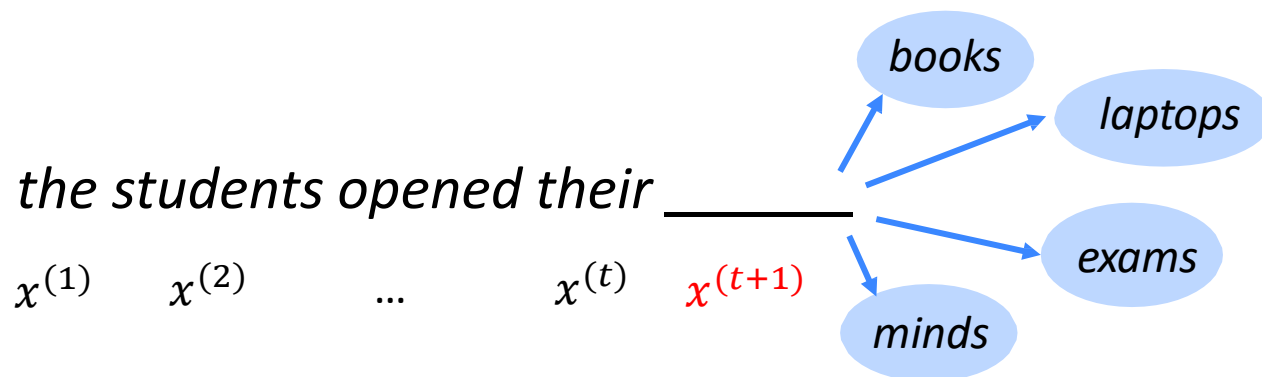
School of Communication and Information

Overview

- Introduce Language Models
 - N-gram Language Model
 - Fixed-window neural language model
- Introduce RNN and RNN Language Model
- Applications of RNN models
- GRU and LSTM models
- Deep Bi-Directional RNN models

Language Modeling

- **Language Modeling** is the task of predicting what word comes next.

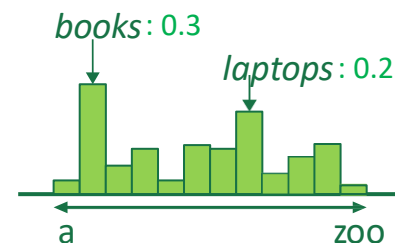


- More formally: given a sequence of words $x^{(1)}, x^{(2)}, \dots, x^{(t)}$, compute the probability distribution of the next word $x^{(t+1)}$:

$$P(x^{(t+1)} = w_j \mid x^{(t)}, \dots, x^{(1)})$$

where w_j is a word in the vocabulary $V = \{w_1, \dots, w_{|V|}\}$

- A system that does this is called a **Language Model**.



You use Language Models every day!



what is the | 

what is the **weather**
what is the **meaning of life**
what is the **dark web**
what is the **xfi**
what is the **doomsday clock**
what is the **weather today**
what is the **keto diet**
what is the **american dream**
what is the **speed of light**
what is the **bill of rights**

- ChatGPT is also built on a Language Model.

n-gram Language Models

the students opened their _____

- **Question**: How to learn a Language Model?
- **Answer** (pre- Deep Learning): learn a *n*-gram Language Model!
- **Definition**: A *n*-gram is a chunk of n consecutive words.
 - *uni*grams: “the”, “students”, “opened”, “their”
 - *bi*grams: “the students”, “students opened”, “opened their”
 - *tri*grams: “the students opened”, “students opened their”
 - 4-grams: “the students opened their”
- **Idea**: Collect statistics about how frequent different n-grams are and use these to predict next word.

n-gram Language Models

- First, we make a **simplifying assumption**: $\mathbf{x}^{(t+1)}$ depends only on the preceding $(n-1)$ words: In a 4-gram model ($n=4$), $n-1=3$.

$$P(\mathbf{x}^{(t+1)} | \mathbf{x}^{(t)}, \dots, \mathbf{x}^{(1)}) = P(\mathbf{x}^{(t+1)} | \overbrace{\mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-n+2)}}^{n-1 \text{ words}}) \quad (\text{assumption})$$

$$\begin{array}{l} \text{prob of a } n\text{-gram} \rightarrow \\ \text{prob of a } (n-1)\text{-gram} \rightarrow \end{array} \quad \begin{array}{l} P(\mathbf{x}^{(t+1)}, \mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-n+2)}) \\ = \\ P(\mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-n+2)}) \end{array} \quad \begin{array}{l} \text{(definition of} \\ \text{conditional prob)} \end{array}$$

- Question:** How do we get these n -gram and $(n-1)$ -gram probabilities?
- Answer:** By **counting** them in some large corpus of text!

$$\approx \frac{\text{count}(\mathbf{x}^{(t+1)}, \mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-n+2)})}{\text{count}(\mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-n+2)})} \quad \begin{array}{l} \text{(statistical} \\ \text{approximation)} \end{array}$$

n-gram Language Models: Example

Suppose we are learning a 4-gram Language Model.

~~as the proctor started the clock, the~~ students opened their _____
discard condition on this

$$P(\mathbf{w}_j | \text{students opened their}) = \frac{\text{count}(\text{students opened their } \mathbf{w}_j)}{\text{count}(\text{students opened their})}$$

In the corpus:

- “students opened their” occurred 1000 times
- “students opened their books” occurred 400 times
 - $\rightarrow P(\text{books} | \text{students opened their}) = 0.4$
- “students opened their exams” occurred 100 times
 - $\rightarrow P(\text{exams} | \text{students opened their}) = 0.1$

Should we have discarded the “proctor” (i.e., invigilator) context?

Problems with n-gram Language Models

Sparsity Problem 1

Problem: What if “students opened their w_j ” never occurred in data? Then w_j has probability 0!

(Partial) Solution: Add small δ to count for every $w_j \in V$. This is called *smoothing*.

Similar to Laplace (Add-1) smoothing in Naïve Bayes.

$$P(w_j | \text{students opened their}) = \frac{\text{count}(\text{students opened their } w_j)}{\text{count}(\text{students opened their})}$$

Sparsity Problem 2

Problem: What if “students opened their” never occurred in data? Then we can’t calculate probability for *any* w_j !

(Partial) Solution: Just condition on “opened their” instead. This is called *backoff*.

$$p(w_j | \text{opened their}) = \frac{\text{count}(\text{open their } w_j)}{\text{count}(\text{open their})}$$

Note: Increasing n makes sparsity problems worse. Typically, we can’t have n bigger than 5.

Storage Problems with n -gram Language Models

Storage: Need to store count for all possible n -grams in the corpus.

$$P(\mathbf{w}_j | \text{students opened their}) = \frac{\text{count}(\text{students opened their } \mathbf{w}_j)}{\text{count}(\text{students opened their})}$$

Increasing n or increasing corpus increases model size!

n-gram Language Models in practice

- We can build a simple **trigram** Language Model over a 1.7-million-word corpus (Reuters) in a few seconds on your laptop

Business and financial news

today the _____

get probability
distribution

$$p(w_j \mid \text{today the}) = \frac{\text{count}(\text{today the } w_j)}{\text{count}(\text{today the})}$$

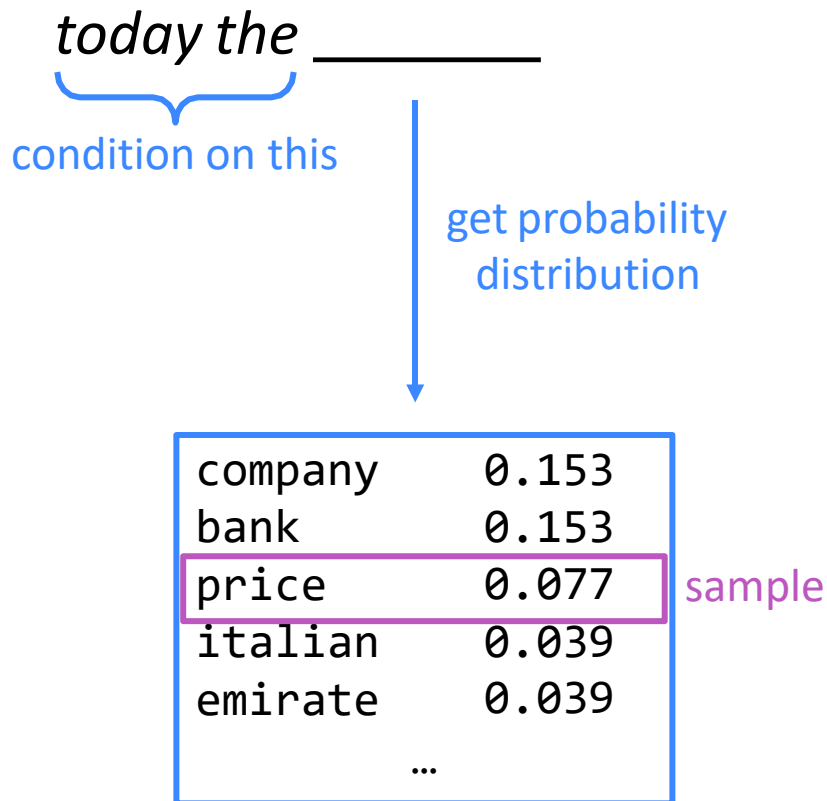
company	0.153
bank	0.153
price	0.077
italian	0.039
emirate	0.039
...	

Sparsity problem:
not much granularity
in the probability
distribution (e.g.,
4/26=0.153)

Otherwise, seems reasonable!

Generating text with a n-gram Language Model

- You can also use a Language Model to **generate text**.

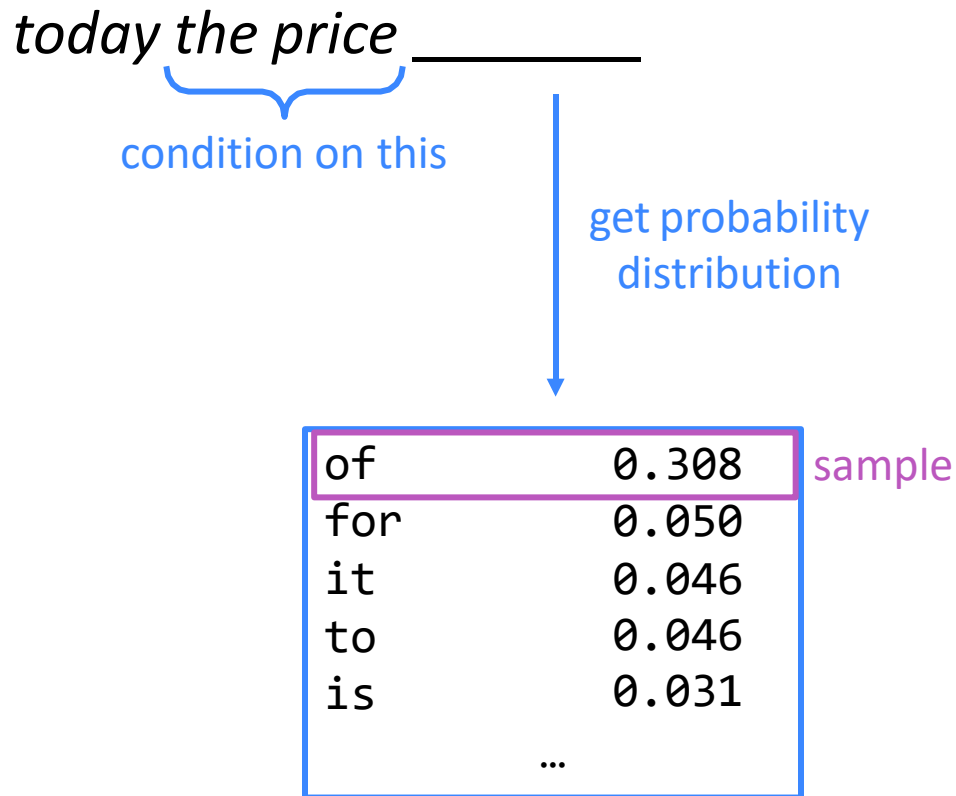


- Sampling is used to generate non-repeating text.
- The next word is selected based on the probability of each word. So “price” is selected with 0.077 probability.

```
next_words = ["company", "bank", "price", "italian", .....]  
probs =      [ 0.153,    0.153, 0.077, 0.039, .....]  
idx_next = torch.multinomial(probs, num_samples=1)  
print(next_words[idx_next]) # price
```

Generating text with a n-gram Language Model

- You can also use a Language Model to **generate text**.



Generating text with a n-gram Language Model

- You can also use a Language Model to **generate text**.

today the price of _____

condition on this

get probability
distribution

the	0.072
18	0.043
oil	0.043
its	0.036
gold	0.018
...	

sample

Generating text with a n-gram Language Model

- You can also use a Language Model to generate text.

today the price of gold _____

Generating text with a n-gram Language Model

- You can also use a Language Model to **generate text**.

today the price of gold per ton , while production of shoe lasts and shoe industry , the bank intervened just after it considered and rejected an imf demand to rebuild depleted european stocks , sept 30 end primary 76 cts a share .

Incoherent! We need to consider more than 3 words at a time if we want to generate good text.

But increasing n worsens sparsity problem, and exponentially increases model size...

A fixed-window neural Language Model

- Recall the Language Modeling task:
 - Input: sequence of words $x^{(1)}, x^{(2)}, \dots, x^{(t)}$
 - Output: prob dist of the next word $P(x^{(t+1)} = w_j \mid x^{(t)}, \dots, x^{(1)})$
- How about a window-based neural model?

~~as the proctor started the clock~~

discard

the students opened their

fixed window

a 5-gram language model

A fixed-window neural Language Model

output distribution

$$\hat{y} = \text{softmax}(U\mathbf{h} + \mathbf{b}_2) \in \mathbb{R}^{|V|}$$

hidden layer

$$\mathbf{h} = f(\mathbf{W}\mathbf{e} + \mathbf{b}_1)$$

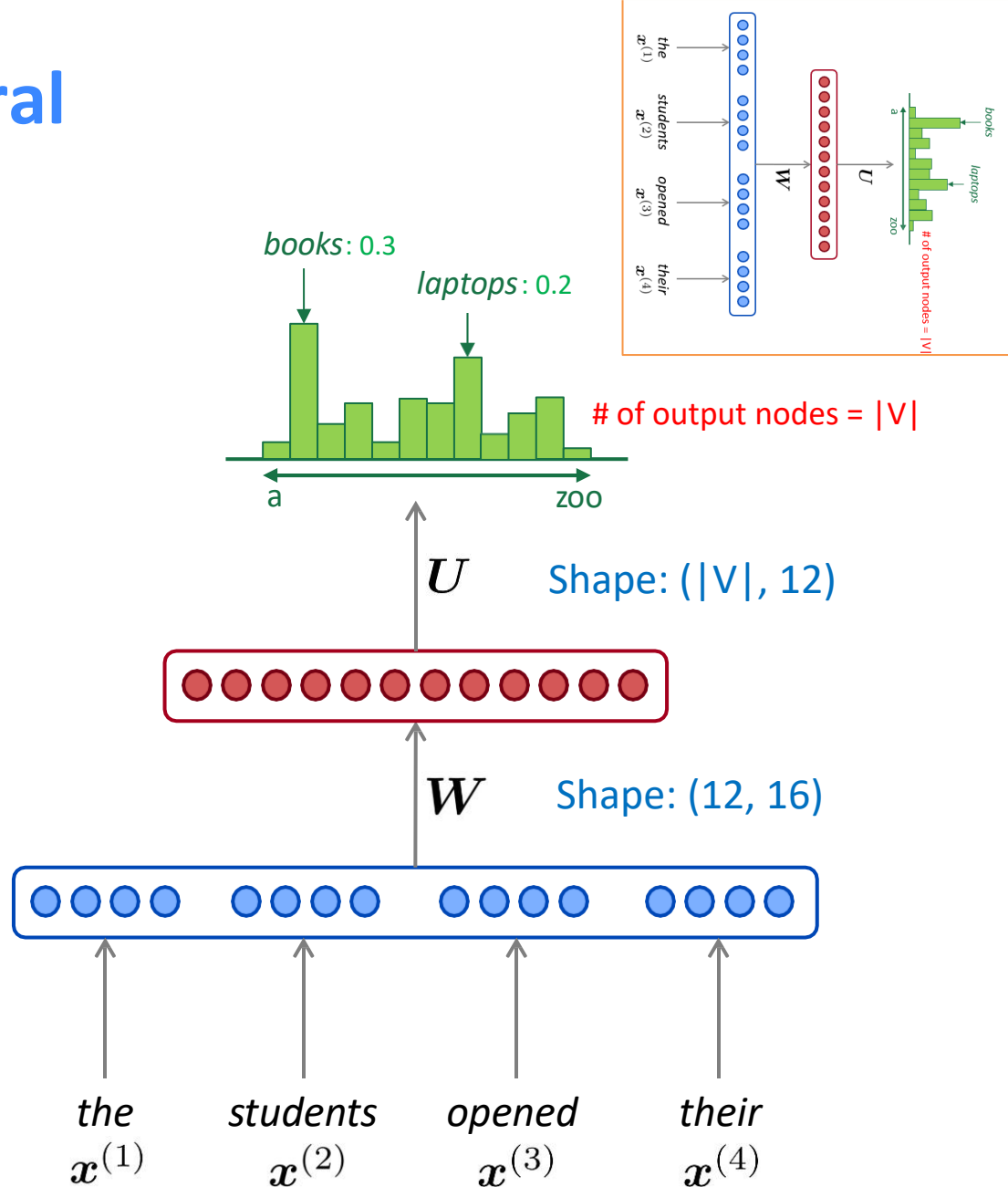
input layer

concatenated word embeddings

$$\mathbf{e} = [\mathbf{e}^{(1)}; \mathbf{e}^{(2)}; \mathbf{e}^{(3)}; \mathbf{e}^{(4)}]$$

words

$$\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \mathbf{x}^{(3)}, \mathbf{x}^{(4)}$$



A fixed-window neural Language Model

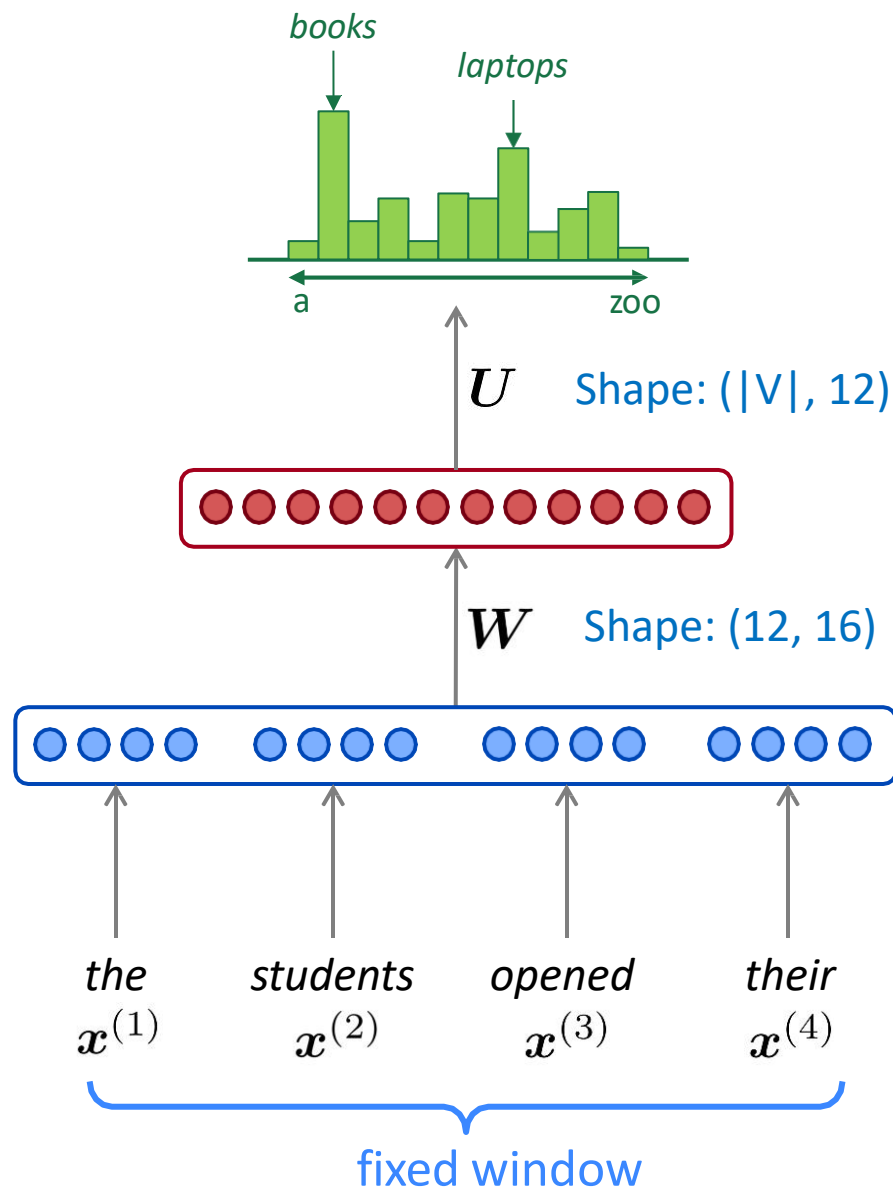
Improvements over n -gram LM:

- No sparsity problem
- Don't need to store all observed n -grams

Remaining **problems**:

- Fixed window is **too small**
- Enlarging window enlarges W
- Window can never be large enough!
- Each $x^{(i)}$ uses different weights of W . We **don't share weights** across the window.

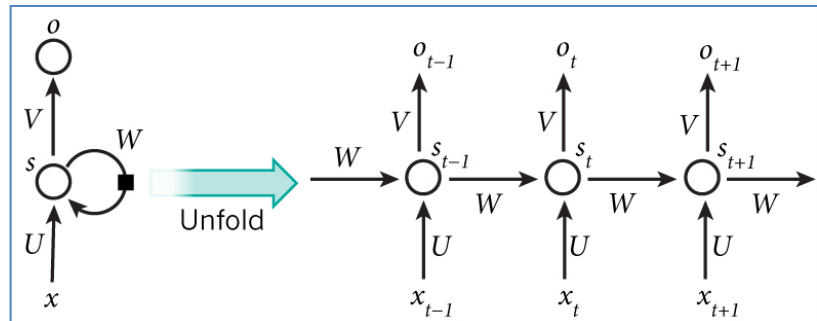
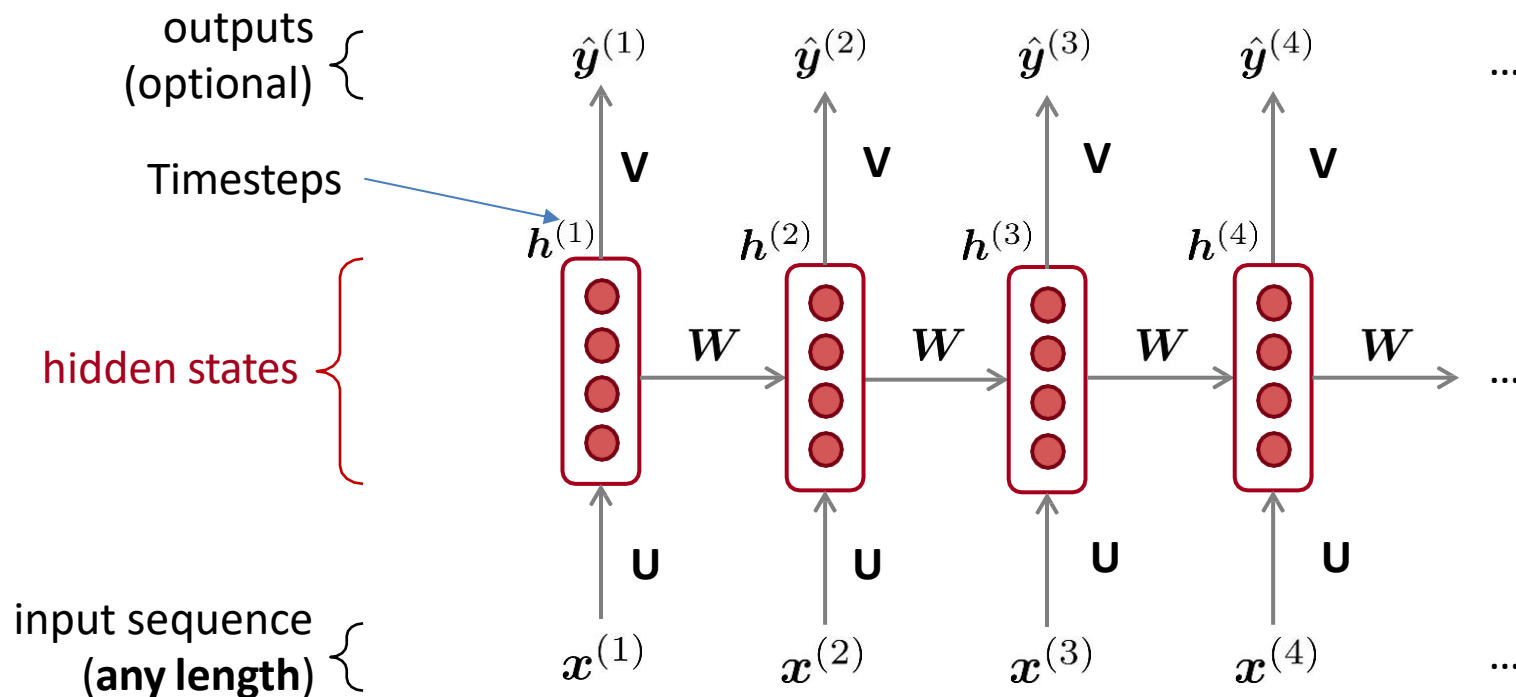
We need a neural architecture that can process *any length input*



Recurrent Neural Networks (RNN)

A family of neural architectures

Core idea: Apply the same weights W repeatedly



A RNN Language Model

output distribution

$$\hat{y}^{(t)} = \text{softmax}(U h^{(t)} + b_2) \in \mathbb{R}^{|V|}$$

hidden states

$$h^{(t)} = \sigma(W_h h^{(t-1)} + W_e e^{(t)} + b_1)$$

$h^{(0)}$ is the initial hidden state
(can be set to 0)

word embeddings

$$e^{(t)} = E(x^{(t)})$$

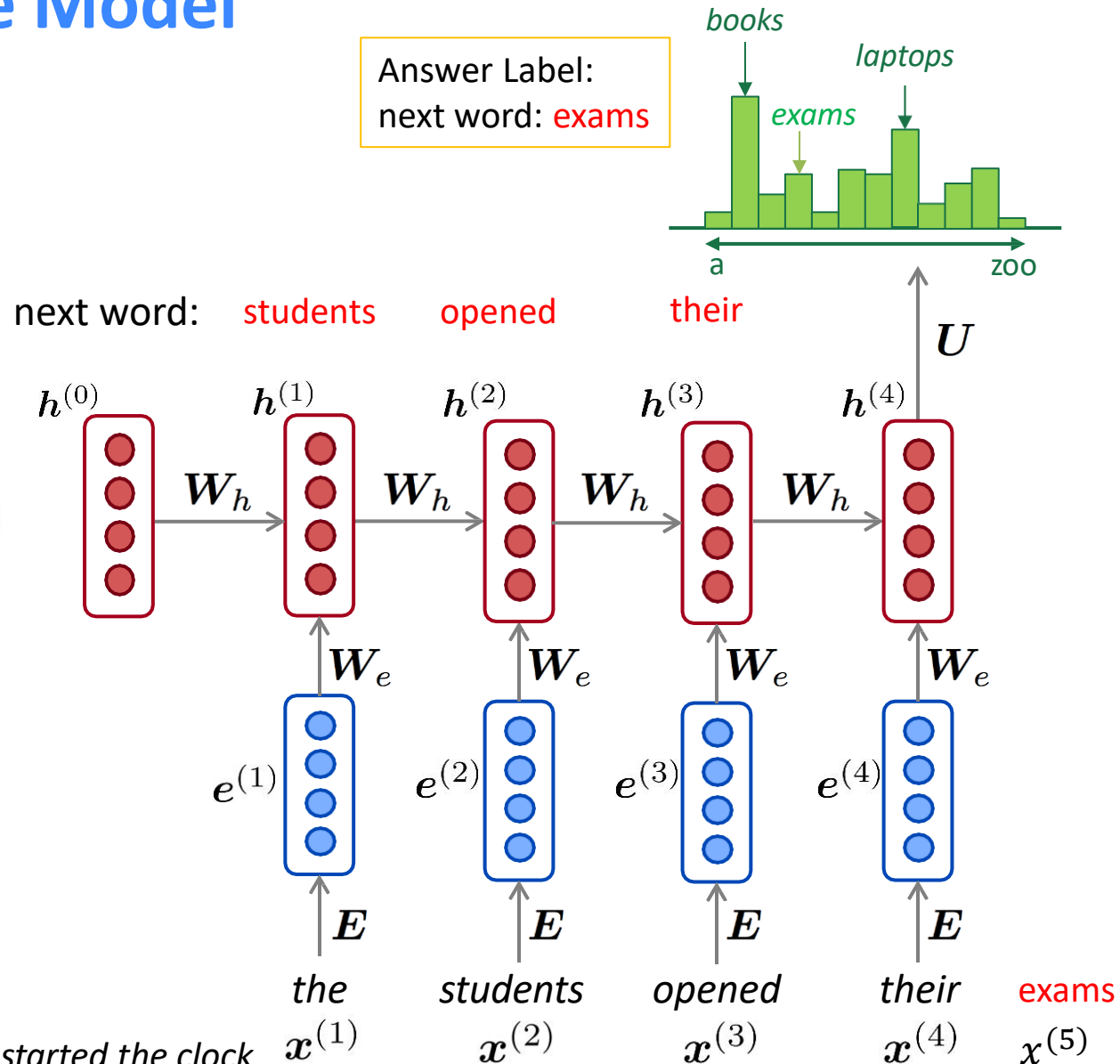
words

$$x^{(t)}$$

As the proctor started the clock

Note: this input sequence could be much longer, but this slide doesn't have space!

$$\hat{y}^{(4)} = P(x^{(5)} | \text{the students opened their})$$



A RNN Language Model

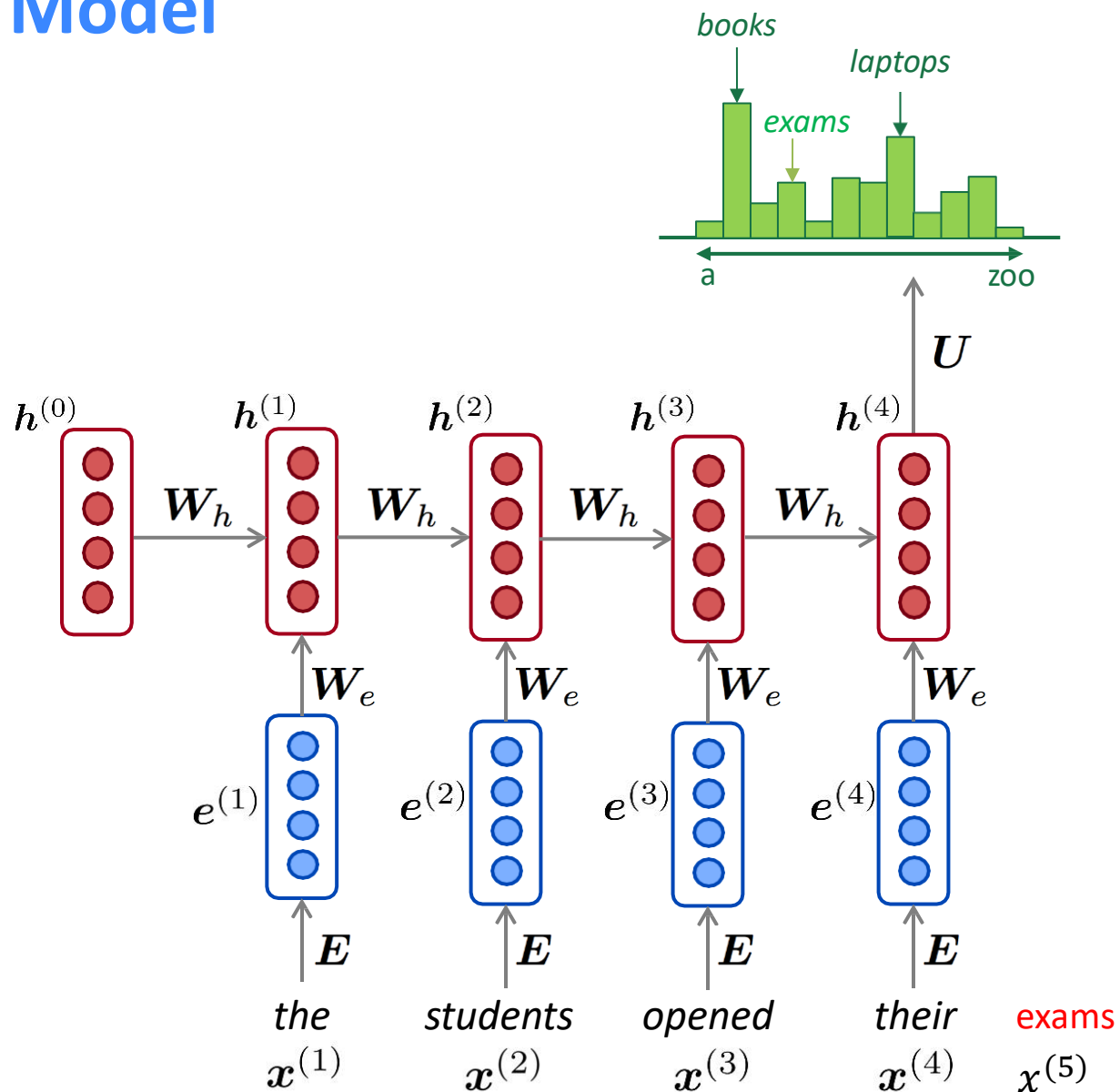
RNN Advantages:

- Can process **any length** input.
- **Model size doesn't increase** for longer input.
- Computation for step t can (in theory) use information from **many steps back**.
- Weights are **shared** across timesteps.

RNN Disadvantages:

- Recurrent computation is **slow**.
- In practice, difficult to access information from **many steps back**.

$$\hat{y}^{(4)} = P(x^{(5)} | \text{the students opened their})$$



Training a RNN Language Model

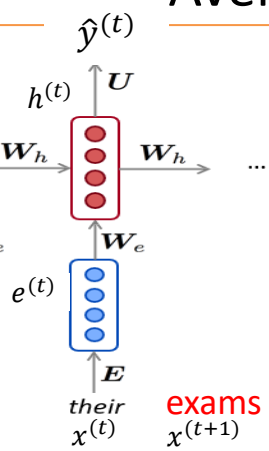
- Get a **big corpus of text** which is a sequence of words: $x^{(1)}, \dots, x^{(T)}$
- Feed into RNN-LM; compute output distribution $\hat{y}^{(t)}$ **for every step t** .
 - i.e., predict probability dist of *every word*, given words so far.
- **Loss function** on step t is **cross-entropy** between predicted probability distribution $\hat{y}^{(t)}$, and the true next word $y^{(t)}$ (one-hot for $x^{(t+1)}$):

$$J^{(t)}(\theta) = CE(y^{(t)}, \hat{y}^{(t)}) = - \sum_{w \in V} y_w^{(t)} \log \hat{y}_w^{(t)} = - \log \hat{y}_{x_{t+1}}^{(t)}$$

when $t=4$,

- Average this to get **overall loss** for entire training set:

$$J(\theta) = \frac{1}{T} \sum_{t=1}^T J^{(t)}(\theta) = \frac{1}{T} \sum_{t=1}^T - \log \hat{y}_{x_{t+1}}^{(t)}$$

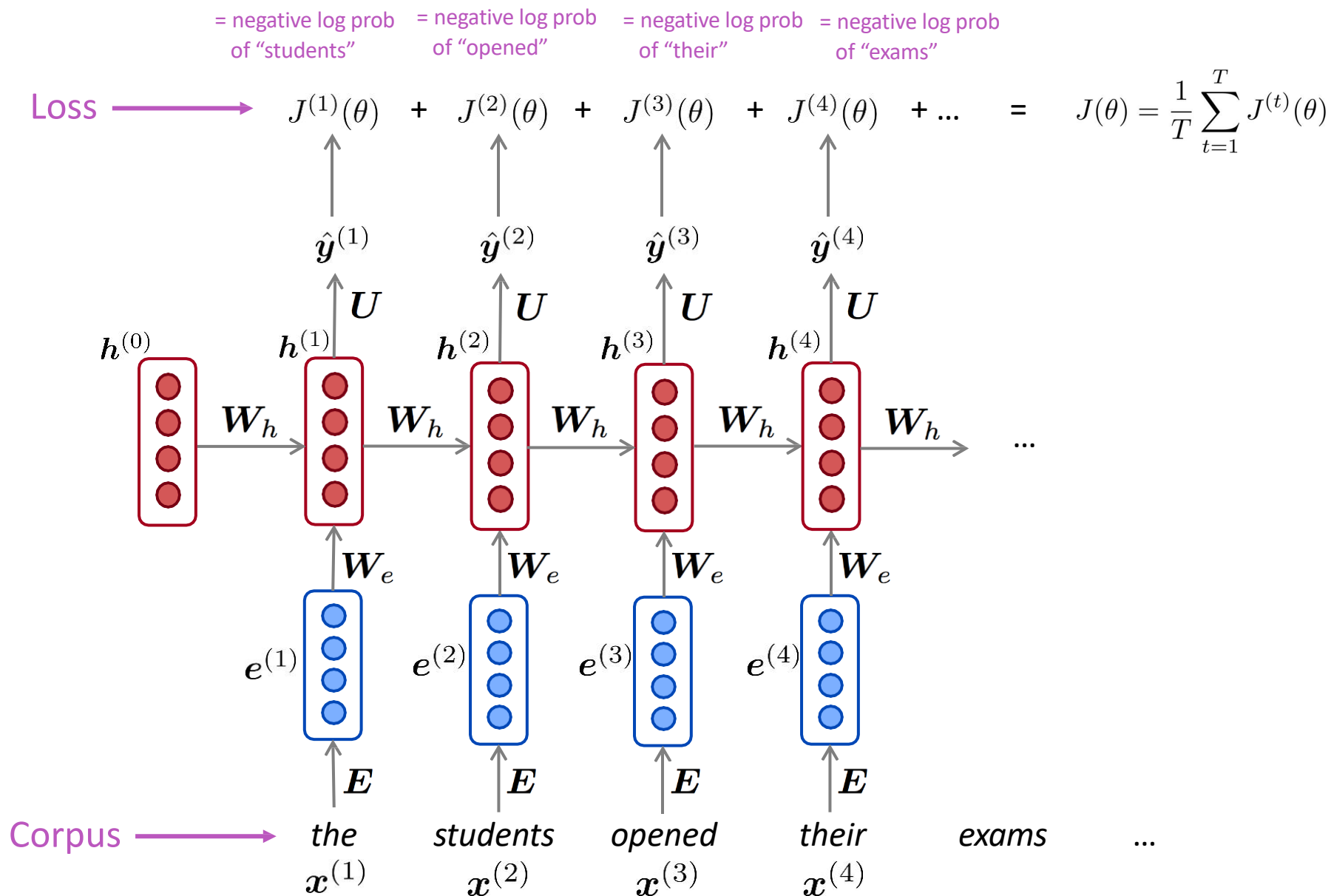


$$y^{(t)} = \begin{matrix} \text{a} \\ \text{exams} \\ \text{zoo} \end{matrix} \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ \dots \\ 0 \end{bmatrix}, \quad \hat{y}^{(t)} = \begin{bmatrix} 0 \\ 0 \\ 0.1 \\ 0 \\ 0 \\ 0.9 \\ 0 \\ \dots \\ 0 \end{bmatrix}$$

= negative log prob
of "exams"
(i.e., $-\log 0.9$)

$$\hat{y}^{(4)} = P(x^{(5)} | \text{the students opened their exams } x^{(4)})$$

Training a RNN Language Model



Training a RNN Language Model

- However: Computing loss and gradients across **entire corpus** is **too expensive!**
- **Stochastic Gradient Descent** allows us to compute loss and gradients for small chunk of data, and update.
 - SGD updates weights per each sample.

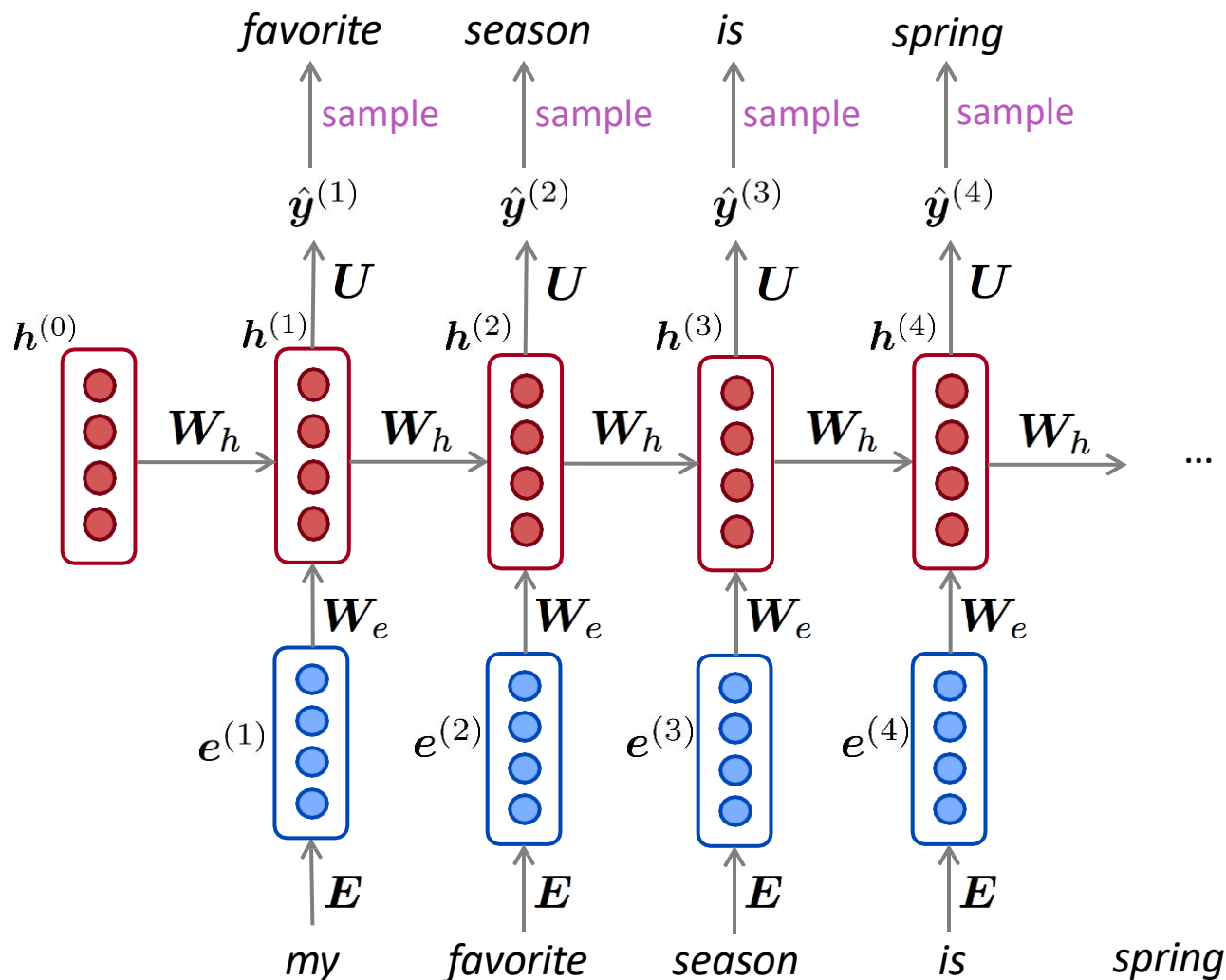
- → In practice, consider $x^{(1)}, \dots, x^{(T)}$ as a **sentence**

$$J(\theta) = \frac{1}{T} \sum_{t=1}^T J^{(t)}(\theta)$$

- Compute loss $J(\theta)$ for a sentence (actually usually a batch of sentences), compute gradients and update weights. Repeat.

Generating text with a RNN Language Model

Just like a n-gram Language Model, you can use a RNN Language Model to **generate text** by **repeated sampling**. Sampled output is next step's input.



Generating text with a RNN Language Model

- You can train an RNN-LM on any kind of text, then generate text in that style.
- RNN-LM trained on Obama speeches:



The United States will step up to the cost of a new challenges of the American people that will share the fact that we created the problem. They were attacked and so that they have to say that all the task of the final days of war that I will not be able to get this done.

Source: <https://medium.com/@samim/obama-rnn-machine-generated-political-speeches-c8abd18a2ea0>

- Better than a n-gram LM, but still not cohesive.
- More advanced language models, such as GPT, can produce more cohesive text.

Evaluating Language Models

- The traditional **evaluation metric** for Language Models is **perplexity**.

$$PP = \prod_{t=1}^T \left(\frac{1}{\sum_{j=1}^{|V|} y_j^{(t)} \cdot \hat{y}_j^{(t)}} \right)^{1/T}$$

Normalized by number of words

Inverse probability of dataset

Without this, PP becomes very big with a big corpus.

When a prediction for a current word is correct, denominator value becomes 1. So, it becomes the smallest value. Otherwise, it will become larger than 1.

- Lower** is better!

True next word

$$y_j^{(t)} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ \mathbf{1} \\ 0 \\ \dots \\ 0 \end{bmatrix}, \hat{y}_j^{(t)} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ \mathbf{1} \\ 0 \\ \dots \\ 0 \end{bmatrix}, \hat{y}_j^{(t)} = \begin{bmatrix} 0.5 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ \mathbf{0.5} \\ 0 \\ \dots \\ 0 \end{bmatrix}$$

RNNs have greatly improved perplexity

n-gram model →

Increasingly complex RNNs ↓

Model	Perplexity
Interpolated Kneser-Ney 5-gram (Chelba et al., 2013)	67.6
RNN-1024 + MaxEnt 9-gram (Chelba et al., 2013)	51.3
RNN-2048 + BlackOut sampling (Ji et al., 2015)	68.3
Sparse Non-negative Matrix factorization (Shazeer et al., 2015)	52.9
LSTM-2048 (Jozefowicz et al., 2016)	43.7
2-layer LSTM-8192 (Jozefowicz et al., 2016)	30
Ours small (LSTM-2048)	43.9
Ours large (2-layer LSTM-2048)	39.8

Perplexity improves (lower is better)

The diagram illustrates the evolution of language models. A purple arrow points from the 'n-gram model' label to the first row of the table. A vertical purple arrow labeled 'Increasingly complex RNNs' points downwards from the first row to the last row. A green box highlights the 'Perplexity' column, and a green arrow points downwards from the top of this box to the text 'Perplexity improves (lower is better)'.

Source: <https://research.fb.com/building-an-efficient-neural-language-model-over-a-billion-words/>

Why should we care about Language Modeling?

- Language Modeling is a **subcomponent** of other NLP systems:
 - Speech recognition
 - Use a LM to generate transcription, **conditioned** on audio
 - “Apple and **P**air Salad” or “Apple and **P**ear Salad”?
 - Machine Translation
 - Use a LM to generate translation, **conditioned** on original text
 - Summarization
 - Use a LM to generate summary, **conditioned** on original text
- Language Modeling is a **benchmark task** that helps us **measure our progress** on understanding language.

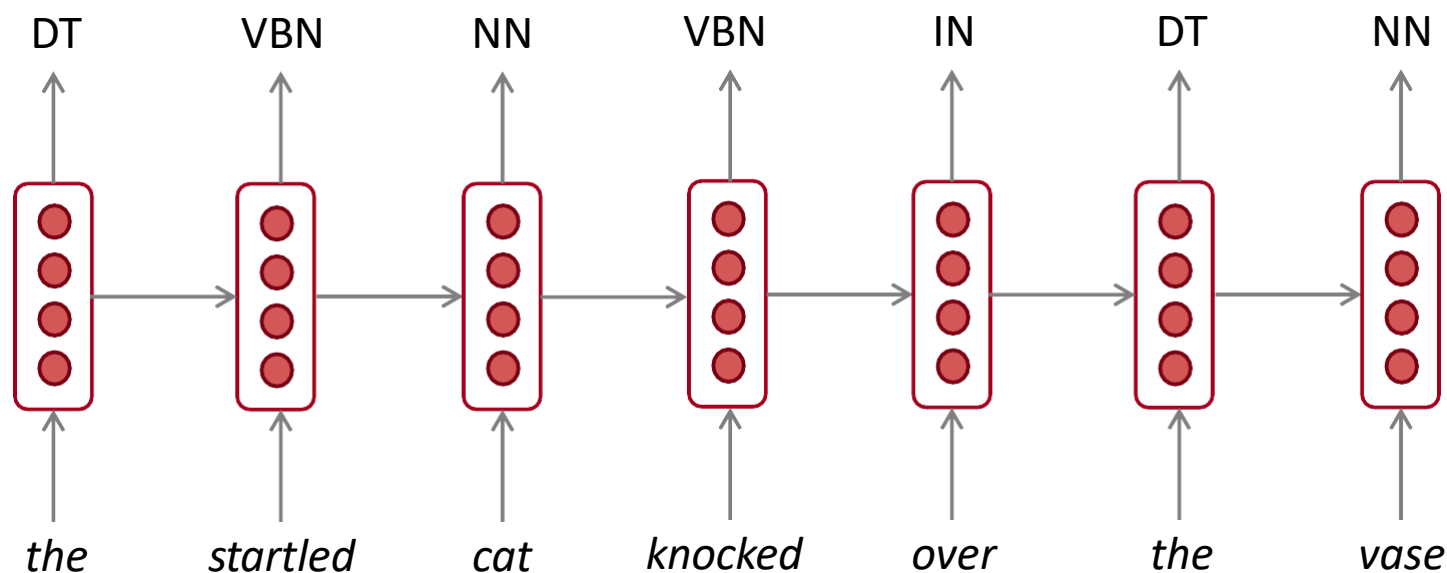
These systems are called *conditional Language Models*

Recap

- Language Model: A system that predicts the next word.
- Recurrent Neural Network: A family of neural networks that:
 - Take sequential input of any length
 - Apply the same weights on each step
 - Can optionally produce output on each step
- Recurrent Neural Network $\neq$ Language Model
- We've shown that RNNs are a great way to build a LM.
- But RNNs are useful for much more!

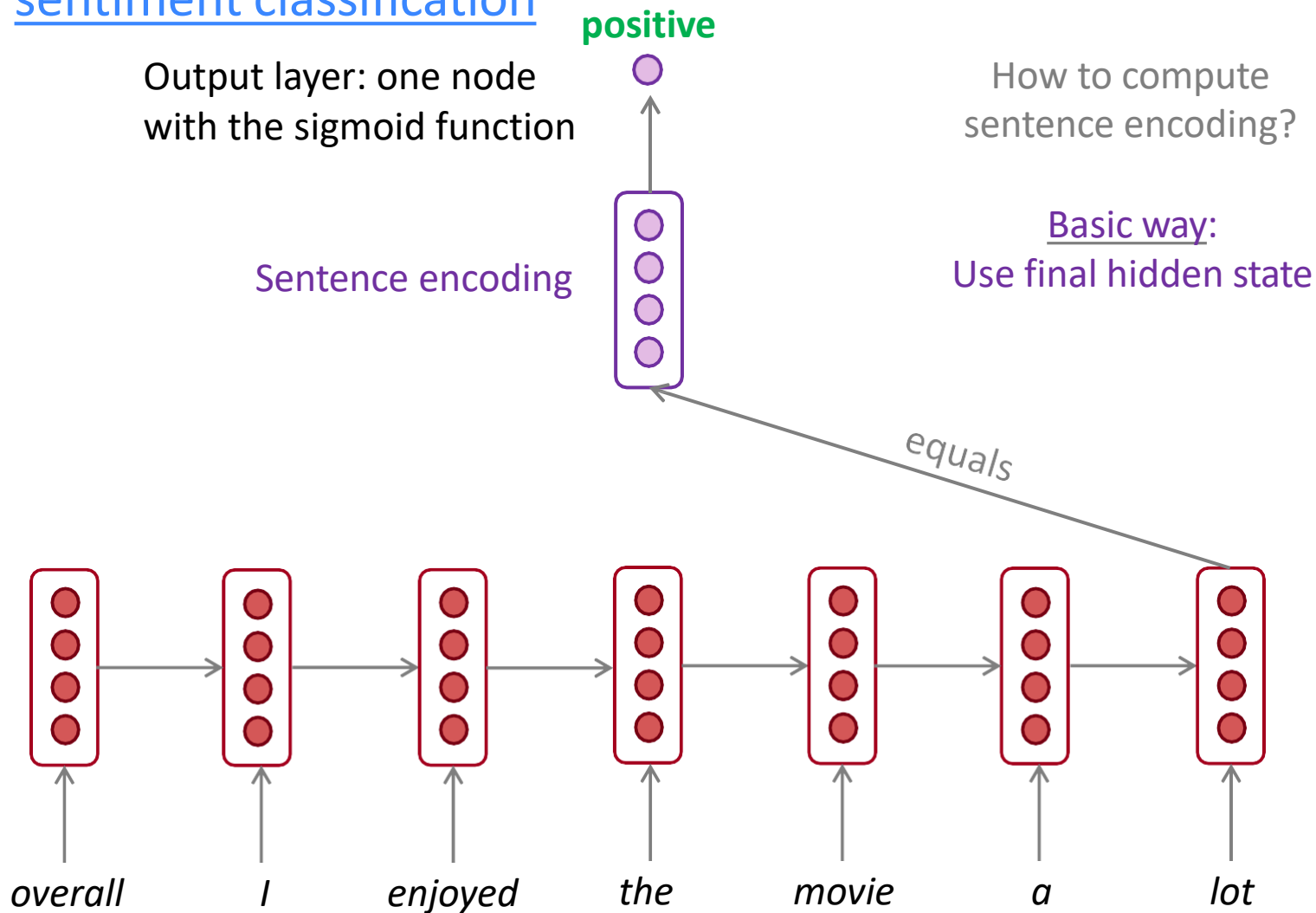
RNNs can be used for tagging

E.g., part-of-speech tagging, named entity recognition



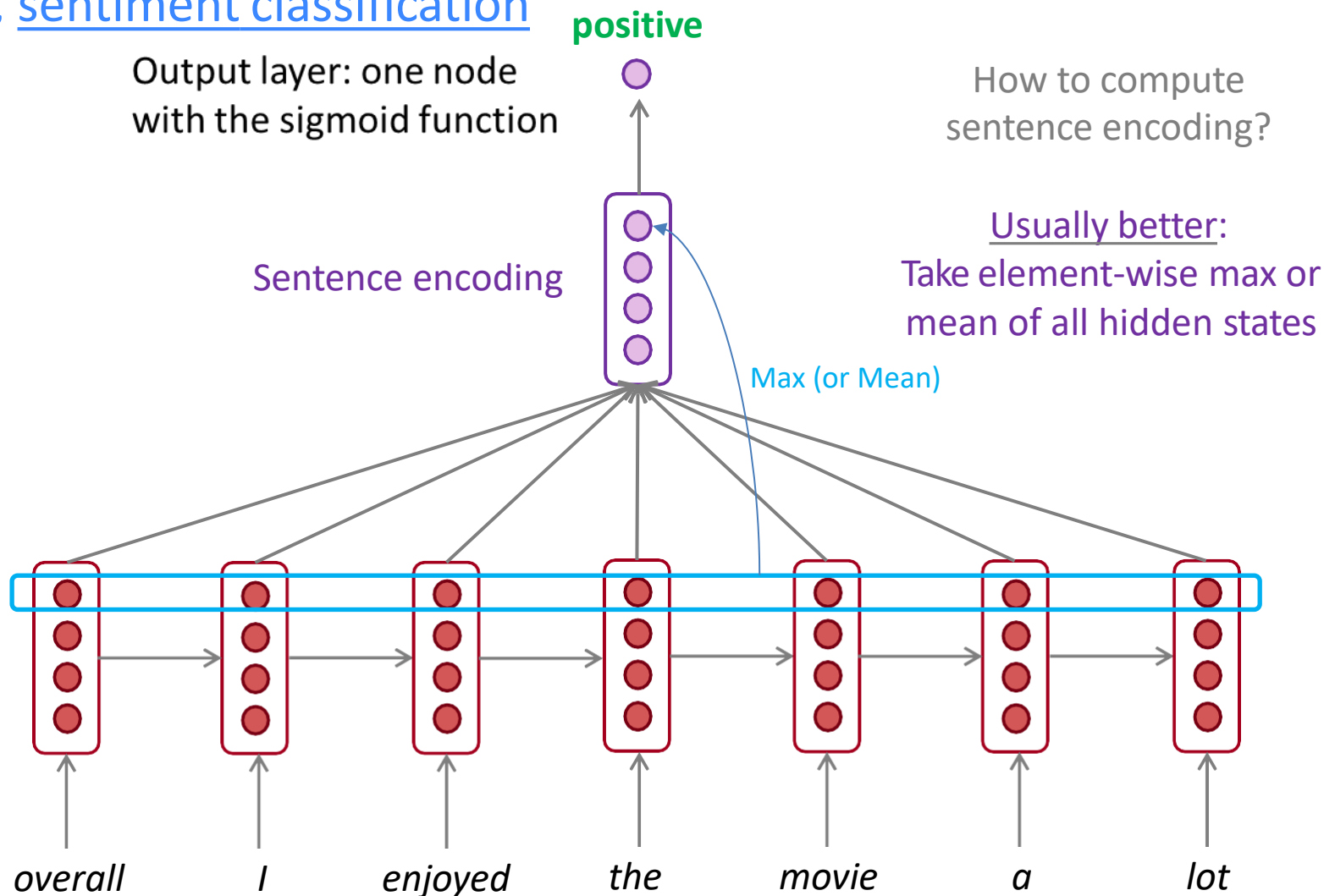
RNNs can be used for sentence classification

E.g., sentiment classification



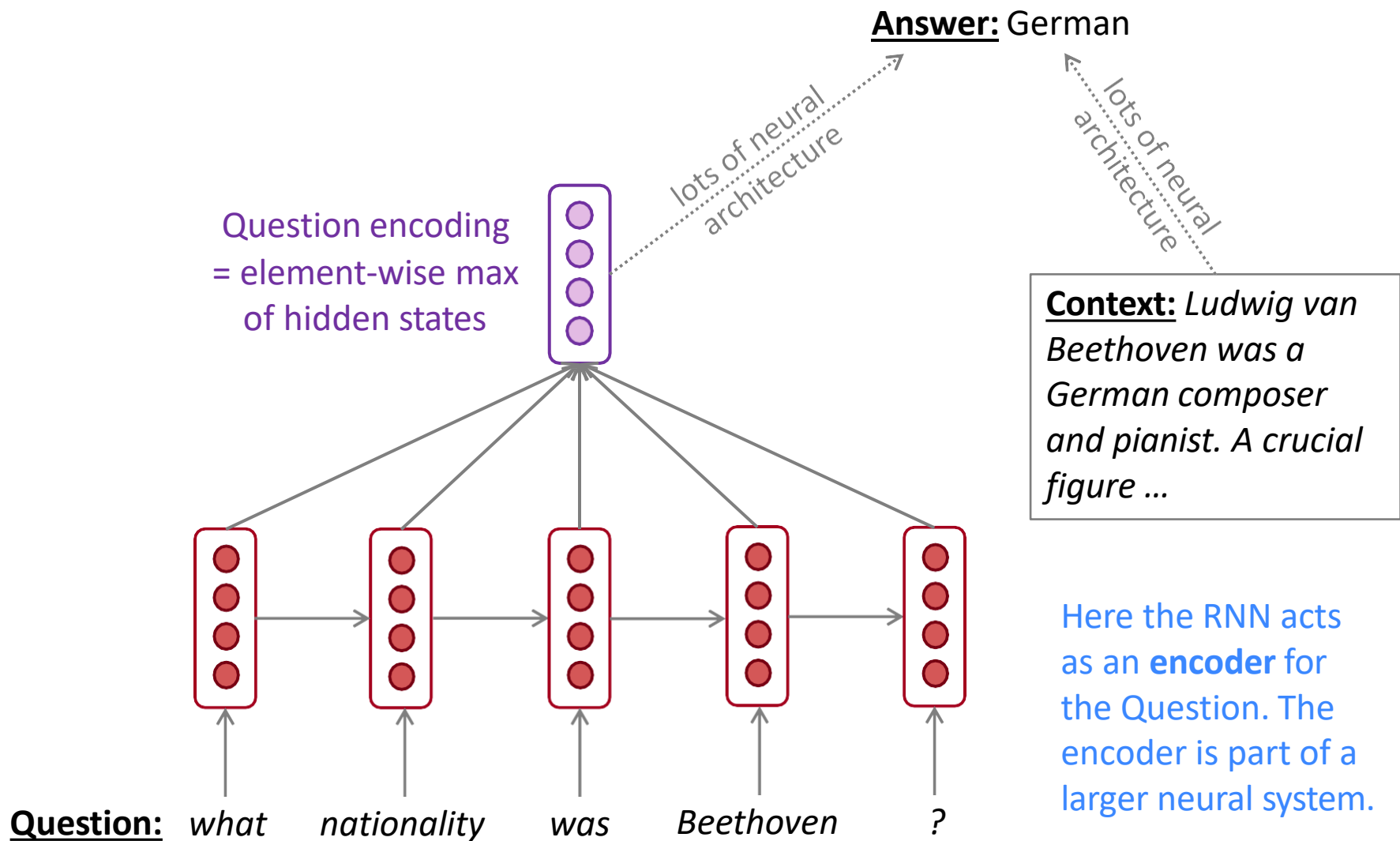
RNNs can be used for sentence classification

E.g., sentiment classification



RNNs can be used as an encoder module

E.g., question answering, machine translation



Problems with RNNs!

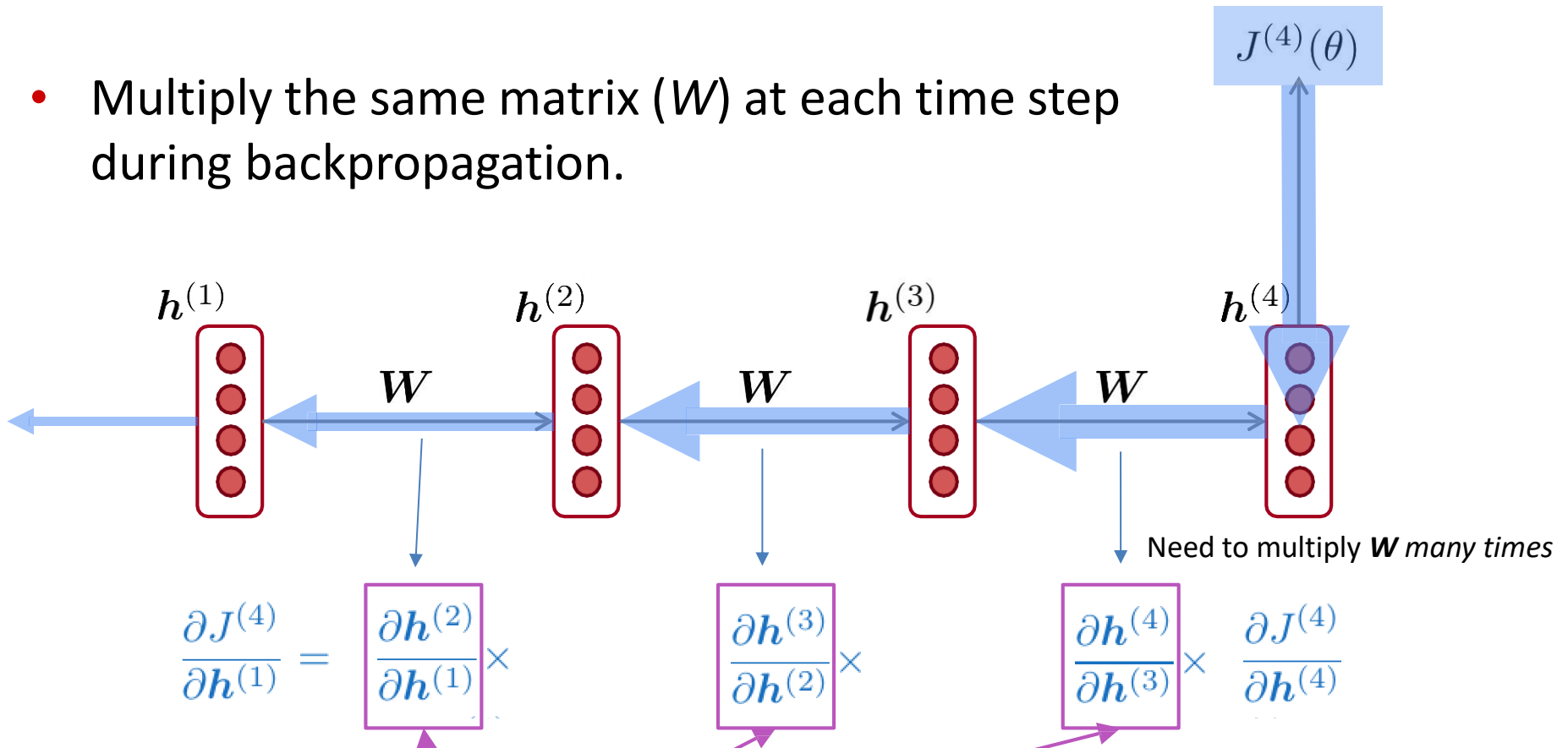
- Problems with RNNs!
 - Vanishing gradients : gradients become very small through backpropagation



- Fancy RNN variants!
 - LSTM
 - GRU
 - multi-layer
 - bidirectional

Vanishing gradient intuition

- Multiply the same matrix (W) at each time step during backpropagation.



What happens if these are small?

Vanishing gradient: current activity is not propagated to the words far from the current one.

Vanishing gradient problem:
When these are small, the gradient signal gets smaller and smaller as it backpropagates further

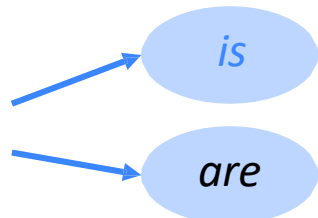
The vanishing gradient problem for language models

- The vanishing gradient problem can cause problems for RNN Language Models:
- When predicting the next word, information from many time steps in the past is not taken into consideration.

- Example:

Jane walked into the room. John walked in too. It was late in the day. Jane said hi to _____

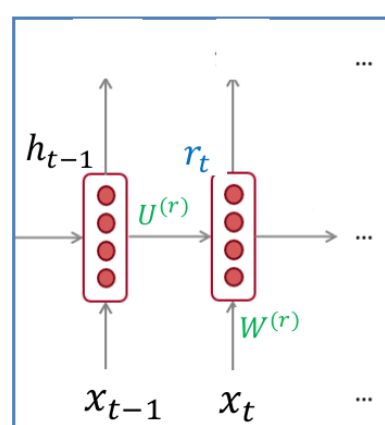
Effect of vanishing gradient on RNN-LM

- **LM task:** *The writer of the books _____*
- **Correct answer:** *The writer of the books is planning a sequel.*
- **Syntactic recency:** *The writer of the books is* (correct)
- **Sequential recency:** *The writer of the books are* (incorrect)
- Due to vanishing gradient, RNN-LMs are better at learning from **sequential recency** than **syntactic recency**, so they make this type of error more often than we'd like [Linzen et al 2016].

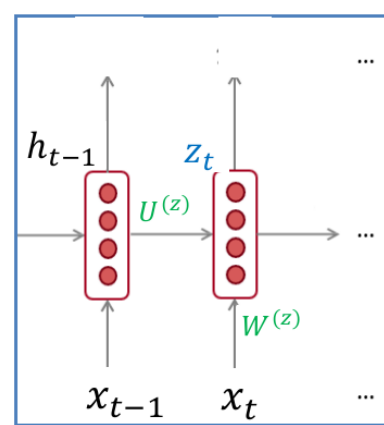
Main solution for better RNNs: Better Units

- The main solution to the Vanishing Gradient Problem is to use a more complex hidden unit computation in recurrence!
- **Gated Recurrent Units (GRU)** introduced by [Cho et al. 2014] and **LSTMs** [Hochreiter & Schmidhuber, 1999]
- Main ideas:
 - Keep around **memories** to capture long distance dependencies.
 - Allow error messages to flow at different strengths depending on the inputs.

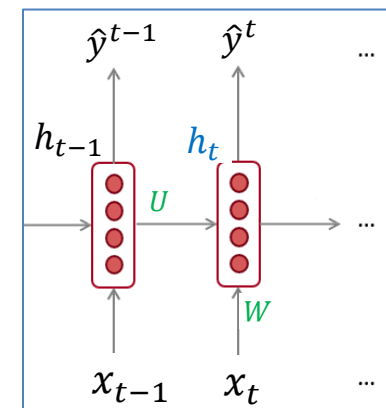
GRUs



Reset Gate



Update Gate



Standard RNN

- Standard RNN computes hidden layer at next time step directly:

$$h_t = \sigma (W x_t + U h_{t-1})$$

- GRU first computes **an update gate** based on current input word vector and hidden state with additional weights

$$z_t = \sigma (W^{(z)} x_t + U^{(z)} h_{t-1})$$

- Compute **reset gate** similarly but with different weights

$$r_t = \sigma (W^{(r)} x_t + U^{(r)} h_{t-1})$$

GRU intuition

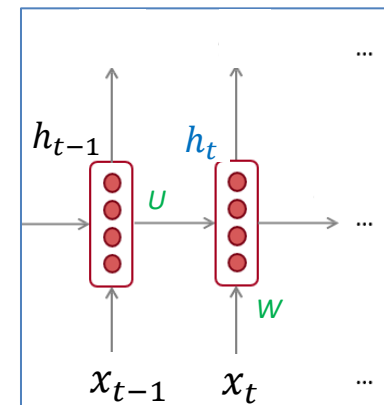
$$h_t = \sigma(Wx_t + Uh_{t-1})$$

- Update gate

$$z_t = \sigma(W^{(z)}x_t + U^{(z)}h_{t-1})$$

- Reset gate

$$r_t = \sigma(W^{(r)}x_t + U^{(r)}h_{t-1})$$

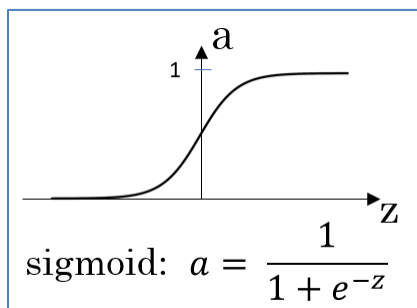


- **New memory content:** $\tilde{h}_t = \tanh(Wx_t + r_t \circ Uh_{t-1})$

If **reset gate** unit is ~ 0 , then this ignores previous memory and only stores the new word information

- **Final memory** at time step combines current and previous time steps:

$$h_t = z_t \circ h_{t-1} + (1 - z_t) \circ \tilde{h}_t$$



$$r_t = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}, U_{t-1} = \begin{bmatrix} 0 \\ 0.7 \\ 0.3 \\ 0.9 \end{bmatrix}, z_t = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 1 \end{bmatrix}, h_{t-1} = \begin{bmatrix} 0 \\ 0.8 \\ 1 \\ 0.9 \end{bmatrix}, \tilde{h}_t = \begin{bmatrix} 0 \\ 0.7 \\ 1 \\ 0.5 \end{bmatrix}$$

Note: Sigmoid is used for update and reset gates.

GRU intuition

- **If reset is close to 0, ignore previous hidden state**
→ Allows model to drop information that is irrelevant in the future.

$$z_t = \sigma \left(W^{(z)} x_t + U^{(z)} h_{t-1} \right)$$

$$r_t = \sigma \left(W^{(r)} x_t + U^{(r)} h_{t-1} \right)$$

$$\tilde{h}_t = \tanh (W x_t + r_t \circ U h_{t-1})$$

$$h_t = z_t \circ h_{t-1} + (1 - z_t) \circ \tilde{h}_t$$

- **Update gate z controls how much of past state should matter now.**
 - **If z close to 1, then we can copy information** in that unit through many time steps! **Less vanishing gradient!**
- Units with short-term dependencies often have reset gates very active.
- Units with long term dependencies have active update gates z.

Long-short-term-memories (LSTMs)

- LSTM is even more complex than GRU

- Allow each time step to modify

- Input gate (current cell matters) $i_t = \sigma \left(W^{(i)} x_t + U^{(i)} h_{t-1} \right)$
- Forget (gate 0, forget past) $f_t = \sigma \left(W^{(f)} x_t + U^{(f)} h_{t-1} \right)$
- Output (how much cell is exposed) $o_t = \sigma \left(W^{(o)} x_t + U^{(o)} h_{t-1} \right)$
- **New memory cell** $\tilde{c}_t = \tanh \left(W^{(c)} x_t + U^{(c)} h_{t-1} \right)$

- Final **memory cell** (stores long-term information): additional output value $c_t = f_t \circ c_{t-1} + i_t \circ \tilde{c}_t$

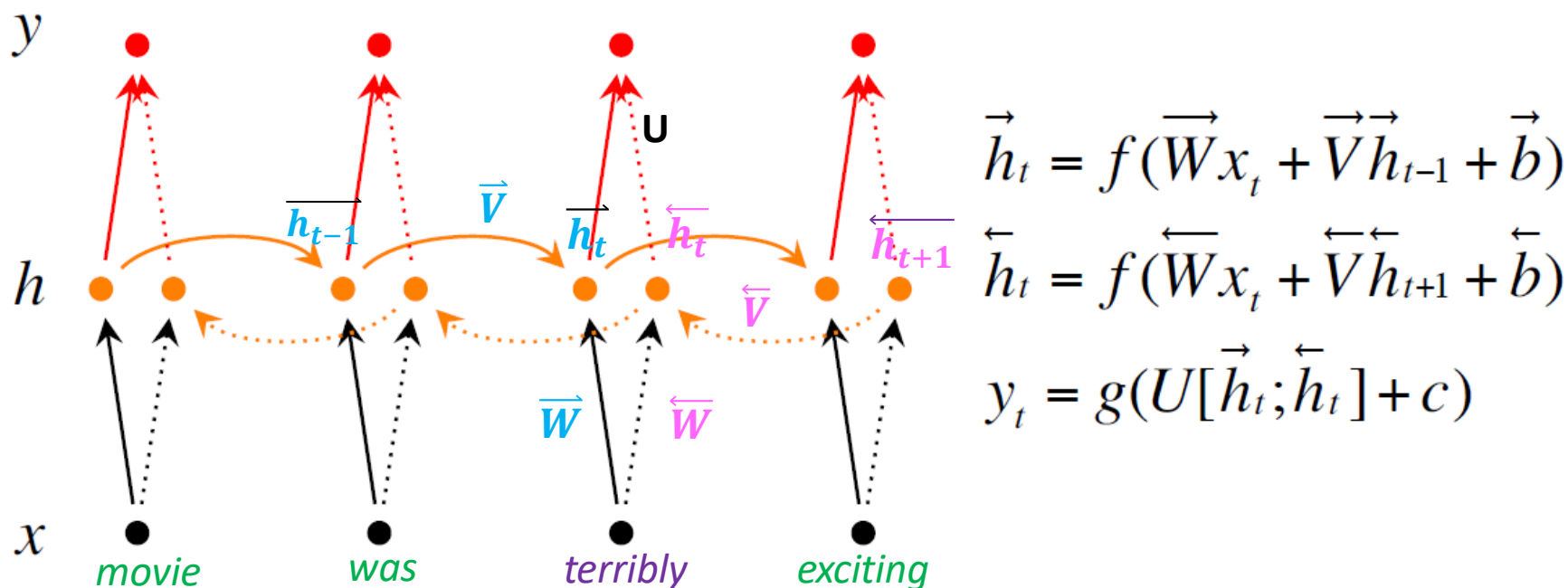
- Final **hidden state**: $h_t = o_t \circ \tanh(c_t)$

LSTMs are a great default for all sequence problems

- Very powerful, especially when stacked and made even deeper (each hidden layer is already computed by a deep internal network)
- Most useful if you have lots and lots of data

Bidirectional RNNs

Problem: For classification, we want to incorporate information from words both preceding and following.



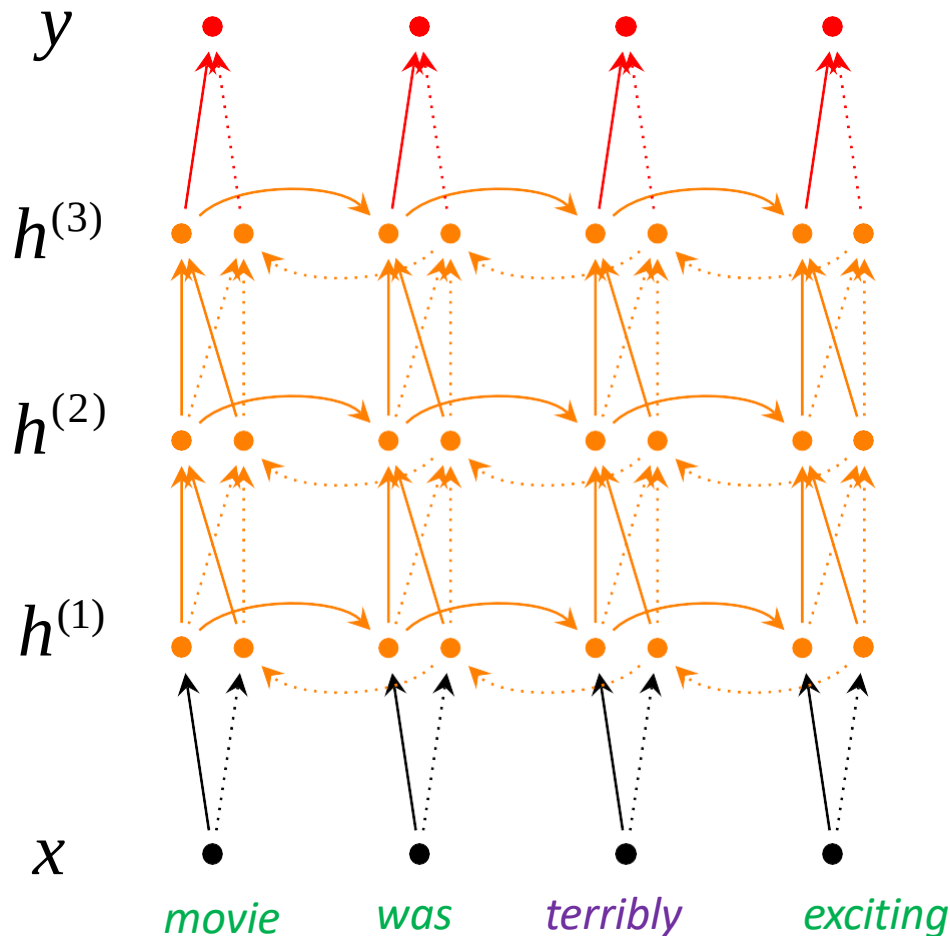
$$\vec{h}_t = f(\vec{W}x_t + \vec{V}\vec{h}_{t-1} + \vec{b})$$

$$\overleftarrow{h}_t = f(\overleftarrow{W}x_t + \overleftarrow{V}\overleftarrow{h}_{t+1} + \overleftarrow{b})$$

$$y_t = g(U[\vec{h}_t; \overleftarrow{h}_t] + c)$$

$h = [\vec{h}; \overleftarrow{h}]$ now represents (summarizes) the past and future around a single token.

Deep Bidirectional RNNs



$$\vec{h}_t^{(i)} = f(\vec{W}^{(i)} h_t^{(i-1)} + \vec{V}^{(i)} \vec{h}_{t-1}^{(i)} + \vec{b}^{(i)})$$

$$\overleftarrow{h}_t^{(i)} = f(\overleftarrow{W}^{(i)} h_t^{(i-1)} + \overleftarrow{V}^{(i)} \overleftarrow{h}_{t+1}^{(i)} + \overleftarrow{b}^{(i)})$$

$$y_t = g(U[\vec{h}_t^{(L)}; \overleftarrow{h}_t^{(L)}] + c)$$

Each memory layer passes an intermediate sequential representation to the next.

Summary of today's lecture

- Learned Language Models
 - N-gram Language Model
 - Fixed-window neural language model
- Learned RNN and RNN Language Model
- Applications of RNN models
- GRU and LSTM models
- Deep Bi-Directional RNN models



Reference

- Natural Language Processing with Deep Learning,
<http://web.stanford.edu/class/cs224n/>
- Sequence Models, Andrew Ng,
<https://www.coursera.org/learn/neural-networks-deep-learning>